

Relatorio de Auditoria de Seguranca

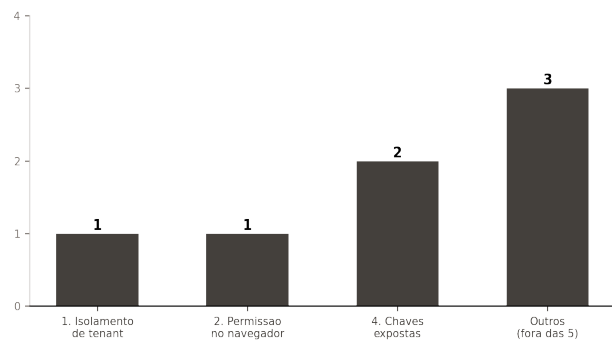
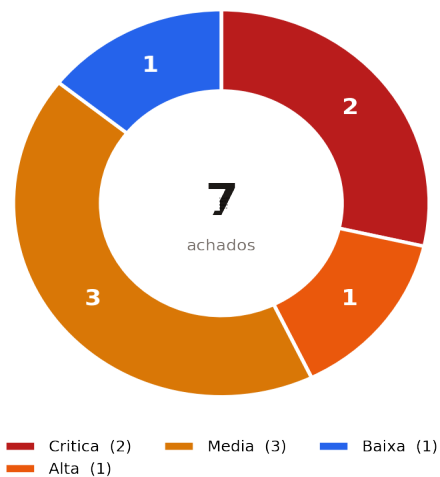
AgroFlux - Gestao Financeira Agricola (SaaS)

Data da auditoria:	11 de setembro de 2026
Escopo:	Codigo-fonte completo do repositorio (front-end React/Vite + funcoes serverless da Vercel + esquema Postgres), 8 rotas de API auditadas linha a linha
Stack detectada:	React + TypeScript + Vite (front-end) Vercel Serverless Functions em Node.js (back-end, sem framework) @vercel/postgres com tagged templates (sem ORM) Autenticacao propria: JWT via 'jose' + bcryptjs, cookie httpOnly

Nota metodologica: cada uma das 5 categorias solicitadas foi mapeada para o equivalente real desta stack antes da analise: (1) isolamento de tenant = coluna familia_id derivada da sessao (JWT) em cada consulta, ja que o projeto nao usa Supabase/RLS; (2) permissao no navegador = comparacao entre os gates de papel/perfil no React e a ausencia ou presenca de checagem equivalente nas funcoes serverless; (3) IDOR = cada handler de rota que aceita um ID de objeto (familiaId, produtoId etc.) foi verificado individualmente; (4) chaves expostas = varredura por segredos hardcoded em todo o codigo-fonte, configs e .env versionado; (5) XSS = varredura por sinks de renderizacao de HTML/JS no front-end React e por geracao de HTML no back-end.

Resumo Executivo

Foram identificados **7 achados** ao longo da auditoria, sendo **2 de severidade critica**. As 8 rotas de API do projeto foram lidas integralmente, linha por linha (nao por amostragem). O achado mais grave e estrutural: o sistema de permissoes por papel (administrador/visualizacao, e as permissoes granulares por acao) existe apenas na interface - o servidor aceita qualquer alteracao de um usuario autenticado da familia, independente do papel dele.



Achados por severidade (esquerda) e por categoria (direita)

Pontos Fortes

O que foi verificado no código e está corretamente protegido:

- **SQL Injection** (api/**/*ts (todas as 8 rotas))

100% das consultas usam tagged templates parametrizados do @vercel/postgres (sql`... \${valor} ...`). Nenhuma concatenação de string em SQL foi encontrada.

- **XSS** (src/**/*tsx (todo o front-end))

Nenhum uso de dangerouslySetInnerHTML, innerHTML, eval ou new Function em todo o código React. Sem sanitização necessária porque não há pontos de inserção de HTML/JS vindos do usuário.

- **Segredos no repositório** (.gitignore, .env.example)

O arquivo .env real está corretamente listado no .gitignore (.env*); apenas o .env.example (sem valores reais) é versionado.

- **Permissão no navegador (rotas administrativas)** (api/admin/familias.ts, api/admin/backup.ts)

exigirMaster() revalida o e-mail do usuário contra a tabela usuarios_auth no banco a cada chamada, em vez de confiar apenas no que o token ou o front-end afirmam - as duas rotas master-only estão corretamente protegidas no servidor.

- **Senhas** (api/auth/signup.ts, api/auth/login.ts)

Senhas são armazenadas apenas como hash bcrypt (custo 10), nunca em texto puro; a comparação usa bcrypt.compare (tempo constante), evitando timing attack trivial.

- **Cookies de sessão** (api/auth/login.ts:24, api/auth/signup.ts:24)

Cookie 'session' configurado com HttpOnly + Secure + SameSite=Lax, reduzindo risco de roubo via XSS (mitigado pela ausência de sinks) e CSRF cross-site.

- **Algoritmo JWT** (todas as rotas de auth)

Uso da lib jose com HS256 explícito; a biblioteca não permite algoritmo 'none', eliminando o vetor clássico de bypass de assinatura.

Pontos Fracos (risco central)

Autorização por papel/permissão existe apenas no front-end (Crítica); segredo de sessão com valor padrão previsível se a variável de ambiente faltar (Crítica); rota de backup automático sem autenticação caso a variável correspondente não esteja configurada (Alta); ausência de proteção contra força bruta e de revogação de sessão (Média).

Achados Detalhados

CRITICA

#1. Endpoint de escrita nao aplica nenhuma regra de papel/permissoao

Categoria: 2. Permissao no navegador

Local: api/familia.ts (arquivo inteiro, linhas 35-83)

O que foi encontrado: O unico endpoint que grava dados de uma familia (PUT /api/familia) aceita e persiste o JSON inteiro enviado pelo cliente sem checar se o usuario autenticado tem permissao para as mudancas contidas nele. As regras de papel existem apenas no front-end: AdministracaoView.tsx:58 (isAdmin), BancosView.tsx:57 (canAdjust), GrupoFamiliarView.tsx:15 (canAdd) e TransacoesView.tsx:58-59 (canDelete/canEdit) escondem botoes na tela, mas nada equivalente existe no servidor.

Por que e explorável / impacto: Qualquer usuario autenticado da familia (mesmo com perfil 'VISUALIZACAO' ou todas as permissoes desmarcadas) pode chamar PUT /api/familia diretamente (fora da UI) com um payload alterado: promover a si mesmo a ADMINISTRADOR, aprovar seu proprio cadastro pendente, apagar lancamentos ou excluir outros usuarios da familia. A unica trava real e a sessao (cookie valido) e o status geral de acesso da familia (bloqueado/pendente) - nao ha checagem por campo ou por acao dentro da familia.

Correcao sugerida: Mover a autorizacao para o servidor: validar no PUT /api/familia quem esta fazendo a chamada (via usuarios_auth + o registro correspondente dentro de dados.usuarios) e recusar (403) alteracoes que a permissao daquele usuario nao permitiria - ou, no minimo, diferenciar endpoints por acao (ex: POST /api/usuarios/aprovar, DELETE /api/despesas/:id) cada um com sua propria checagem de papel no servidor, em vez de um unico PUT que aceita o blob inteiro.

CRITICA

#2. Segredo padrao do JWT hardcoded no codigo-fonte (fallback publico)

Categoria: 4. Chaves expostas

Local: api/familia.ts:6, api/auth/me.ts:6, api/auth/login.ts:7, api/auth/signup.ts:7, api/admin/familias.ts:6, api/admin/backup.ts:6

O que foi encontrado: As 6 funcoes que assinam ou verificam o cookie de sessao usam:

```
const secret = new TextEncoder().encode(process.env.JWT_SECRET ||  
'troque-esta-chave-antes-de-ir-para-producao');
```

Essa string literal esta commitada no repositorio Git (privado ou nao).

Por que e explorável / impacto: Se a variavel de ambiente JWT_SECRET nao estiver definida em QUALQUER ambiente de deploy (producao, preview, uma branch nova, um fork, um ambiente de teste local), o segredo real passa a ser essa string publica. Qualquer pessoa com acesso ao codigo-fonte consegue forjar um cookie 'session' valido com qualquer uid/familiald - inclusive se passando pelo usuario master, ganhando acesso a todos os clientes.

Correcao sugerida: Remover o fallback. Se JWT_SECRET nao existir, a funcao deve lancar erro e recusar iniciar (fail-closed), nunca operar com um segredo previsivel. Centralizar a leitura do segredo em um unico modulo (ver achado de manutenibilidade abaixo) para nao repetir o mesmo bug em 6 lugares.

ALTA

#3. Rota de backup automatico fica sem autenticao se CRON_SECRET nao estiver configurado

Categoria: 4. Chaves expostas (fail-open)

Local: api/cron/backup.ts:10

O que foi encontrado:

```
if (process.env.CRON_SECRET && auth !== `Bearer ${process.env.CRON_SECRET}`) {  
... 401 }
```

Quando CRON_SECRET nao esta definido, a primeira metade do 'if' e falsa e a checagem inteira e pulada - a rota responde normalmente sem exigir nenhuma autenticao.

Por que e explorável / impacto: Qualquer pessoa na internet pode chamar POST/GET em /api/cron/backup e disparar a rotina de snapshot de todos os clientes sob demanda (nao vaza dados diretamente, pois nao retorna os dados, mas permite acionar carga no banco a vontade e cria linhas na tabela backups sem controle).

Correcao sugerida: Inverter a logica para fail-closed: recusar a chamada (401) sempre que CRON_SECRET nao estiver definido, em vez de permitir. Confirmar tambem que a variavel esta configurada em producao.

MEDIA

#4. Escrita usa o familiald do token (JWT) em vez de re-derivar do banco

Categoria: 1. Banco sem tranca (isolamento de tenant)

Local: api/familia.ts:43-76

O que foi encontrado: A consulta de status/limite (linhas 43-48) busca os dados atuais da familia via JOIN por usuarios_auth.id = sessao.uid, mas o UPDATE final (linha 75) usa sessao.familiald - o valor cru do token - em vez do familia_id devolvido nessa mesma consulta.

Por que e explorável / impacto: Hoje nao ha nenhuma funcionalidade que reatribua um usuario a outra familia, entao nao e explorada na pratica. Mas e uma inconsistencia de projeto: se essa funcionalidade for criada no futuro sem revisar este ponto, um usuario movido de familia continuaria escrevendo na familia antiga enquanto seu token de 30 dias nao expirar.

Correcao sugerida: Usar sempre o familia_id retornado pela consulta ao banco (fresco) como fonte da verdade para autorizacao de escrita, nunca o claim do token isoladamente.

MEDIA

#5. Sem limite de tentativas de login (forca bruta de senha)

Categoria: Fora das 5 categorias - Autenticacao

Local: api/auth/login.ts (arquivo inteiro)

O que foi encontrado: O endpoint de login compara a senha via bcrypt normalmente, mas nao ha nenhum mecanismo de rate limiting, atraso progressivo ou bloqueio temporario por tentativas erradas repetidas para o mesmo e-mail ou IP.

Por que e explorável / impacto: Um atacante pode tentar senhas em sequencia (forca bruta ou lista de senhas vazadas) contra qualquer e-mail conhecido sem restricao automatica, limitado apenas por eventuais protecoes de infraestrutura da Vercel (nao garantidas a nivel de aplicacao).

Correcao sugerida: Adicionar limite de tentativas por e-mail/IP (ex: usando a propria tabela Postgres ou um servico de rate limit), com bloqueio temporario progressivo apos poucas tentativas.

MEDIA

#6. Sessao de 30 dias sem mecanismo de revogacao no servidor

Categoria: Fora das 5 categorias - Gestao de sessao

Local: api/auth/login.ts:22, api/auth/signup.ts:22, api/auth/logout.ts

O que foi encontrado: Os tokens JWT sao emitidos com setExpirationTime('30d') e o logout apenas apaga o cookie no navegador (api/auth/logout.ts) - o token em si continua criptograficamente valido ate expirar, mesmo depois do logout.

Por que e explorável / impacto: Se um token vazar (por exemplo, roubo de cookie via dispositivo comprometido), ele permanece utilizavel por ate 30 dias mesmo que a vitima faca logout ou troque a senha - nao existe lista de revogacao nem versao de sessao checada no servidor.

Correcao sugerida: Reduzir o tempo de vida do token, e/ou guardar uma versao de sessao por usuario no banco (incrementada no logout ou na troca de senha) que e checada em toda verificacao do token, invalidando tokens antigos imediatamente quando necessario.

#7. Campos do PUT administrativo aceitos sem validacao de tipo/enum

Categoria: Fora das 5 categorias - Validacao de entrada

Local: api/admin/familias.ts:73-87

O que foi encontrado: status, limiteUsuarios e plano vindos de req.body sao gravados via COALESCE sem checar se status pertence ao conjunto esperado ('pendente'|'ativo'|'bloqueado') ou se limiteUsuarios e um numero positivo.

Por que e explorável / impacto: Como o endpoint ja exige perfil master (exigirMaster confirmado como correto), o risco e baixo - trata-se de um usuario ja confiavel podendo gravar um valor inconsistente por engano, nao uma escalada de privilegio.

Correcao sugerida: Validar status contra a lista de valores permitidos e limiteUsuarios como inteiro positivo antes do UPDATE, retornando 400 em caso de valor invalido.

Recomendacoes Priorizadas

P1 - Fazer antes de qualquer venda comercial

- Remover o valor padrao hardcoded do JWT_SECRET nos 6 arquivos; falhar a inicializacao se a variavel nao existir.
- Corrigir api/cron/backup.ts para negar acesso (fail-closed) quando CRON_SECRET nao estiver definido.
- Adicionar checagem de papel/permisao no servidor antes de aceitar escritas sensiveis em /api/familia (aprovar usuario, mudar perfil, excluir lancamento).

P2 - Fazer no proximo ciclo

- Adicionar rate limiting no login (por e-mail e por IP).
- Implementar revogacao de sessao (versao de sessao no banco) e reduzir a validade do token.
- Usar o familia_id retornado pela consulta ao banco, nao o claim do JWT, na escrita de /api/familia.

P3 - Melhoria continua

- Validar tipos/enum dos campos administrativos recebidos em api/admin/familias.ts.
- Extrair a logica de secret/verificacao de sessao para um unico modulo compartilhado, para evitar que a mesma falha se repita em varios arquivos no futuro.

Issues para o GitHub

Texto pronto para colar em issues do GitHub, uma por bloco delimitado.

--- ISSUE 1 ---

[Seguranca] Segredo de JWT com valor padrao hardcoded (6 arquivos)

Labels: security, critica

Problema

As funcoes de autenticacao usam `process.env.JWT\_SECRET || 'troque-esta-chave-antes-de-ir-para-producao'`.

Se a variavel de ambiente nao estiver definida em algum ambiente de deploy, esse valor publico vira o segredo real usado para assinar e verificar os cookies de sessao.

Evidencia

- api/familia.ts:6
- api/auth/me.ts:6
- api/auth/login.ts:7
- api/auth/signup.ts:7
- api/admin/familias.ts:6
- api/admin/backup.ts:6

```
``ts
const secret = new TextEncoder().encode(
process.env.JWT_SECRET || 'troque-esta-chave-antes-de-ir-para-producao'
);
``
```

Impacto

Qualquer pessoa com acesso ao codigo-fonte pode forjar um token de sessao valido para qualquer usuario, incluindo o master, se o ambiente rodar sem JWT_SECRET configurado.

Sugestao de correcao

Remover o fallback. Lancar erro na inicializacao se `process.env.JWT\_SECRET` nao existir. Centralizar essa logica em um unico modulo importado pelos 6 arquivos.

Critérios de aceite

- [] Nenhum arquivo contem um valor de segredo literal como fallback
- [] A aplicacao recusa iniciar/responder se JWT_SECRET nao estiver definido
- [] Testado localmente removendo a variavel de ambiente e confirmando a falha controlada

--- FIM ISSUE 1 ---

--- ISSUE 2 ---

[Seguranca] Rota de backup automatico sem autenticacao quando CRON_SECRET nao esta definido

Labels: security, alta

Problema

Em `api/cron/backup.ts`, a checagem de autenticacao so roda se `CRON\_SECRET` existir:

```
``ts
if (process.env.CRON_SECRET && auth !== `Bearer ${process.env.CRON_SECRET}`) {
res.status(401).json({ error: 'Nao autorizado.' });
return;
}
``
```

Se a variavel nao estiver configurada, a condicao inteira e falsa e a rota fica publica.

Evidencia

api/cron/backup.ts:10

Impacto

Qualquer pessoa na internet pode chamar a rota e disparar a geracao de snapshots de todos os clientes sob demanda.

```

## Sugestao de correcao
Inverter a logica para fail-closed: recusar sempre que `CRON_SECRET` nao estiver definido.

## Critérios de aceite
- [ ] A rota retorna 401 quando CRON_SECRET nao esta definido
- [ ] A rota so aceita chamadas com o header Authorization correto quando o secret existe
- [ ] CRON_SECRET confirmado como configurado em producao
--- FIM ISSUE 2 ---

--- ISSUE 3 ---
# [Seguranca] Permissoes de papel nao sao verificadas no servidor (api/familia.ts)

**Labels:** security, critica

## Problema
O front-end esconde botoes com base no papel do usuario (AdministracaoView.tsx:58, BancosView.tsx:57, GrupoFamiliarView.tsx:15, TransacoesView.tsx:58-59), mas o unico endpoint de escrita (`PUT /api/familia`) aceita e grava qualquer JSON enviado por um usuario autenticado da familia, sem checar se a acao contida ali e permitida para o papel/permissoes daquele usuario.

## Evidencia
api/familia.ts, linhas 35-83 (arquivo inteiro)

## Impacto
Um usuario com perfil 'VISUALIZACAO' ou com todas as permissoes desmarcadas pode, chamando a API diretamente (fora da interface), promover a si mesmo a ADMINISTRADOR, aprovar seu proprio acesso, excluir outros usuarios ou apagar/alterar qualquer lancamento financeiro da familia.

## Sugestao de correcao
Adicionar verificacao de papel/permisao no servidor antes de persistir mudancas sensiveis, ou dividir o endpoint unico em rotas especificas por acao, cada uma com sua propria checagem de autorizacao no back-end.

## Critérios de aceite
- [ ] Uma requisicao de um usuario sem permisao de administrador e rejeitada ao tentar alterar papeis/aprovar usuarios
- [ ] Uma requisicao de um usuario sem permisao de exclusao e rejeitada ao tentar remover lancamentos
- [ ] Testes automatizados cobrindo pelo menos os 4 gates de papel identificados no front-end
--- FIM ISSUE 3 ---

--- ISSUE 4 ---
# [Seguranca] Escrita em /api/familia usa familiaId do token em vez do banco

**Labels:** security, media

## Problema
O UPDATE final usa `sessao.familiaId` (valor do JWT) em vez do `familia_id` retornado pela consulta que ja foi feita ao banco (via JOIN por `usuarios_auth.id = sessao.uid`).

## Evidencia
api/familia.ts:43-48 (consulta) e linha 75 (uso de sessao.familiaId no UPDATE)

## Impacto
Nao explorado hoje (nao existe funcionalidade de mover usuario entre familias), mas e uma inconsistencia de projeto que pode causar escrita na familia errada se essa funcionalidade for adicionada sem revisar este ponto.

## Sugestao de correcao
Extrair o `familia_id` da propria consulta ja feita ao banco e usa-lo no UPDATE, em vez do claim do token.

## Critérios de aceite
- [ ] O UPDATE usa o familia_id vindo da consulta ao banco, nao do payload do JWT
- [ ] Comentario no codigo explicando por que essa fonte e preferida
--- FIM ISSUE 4 ---

--- ISSUE 5 ---

```

```
# [Seguranca] Sem rate limiting no login e sem revogacao de sessao

**Labels:** security, media

## Problema
`api/auth/login.ts` nao limita tentativas de senha por e-mail/IP. Os tokens JWT tem validade de 30 dias
(`api/auth/login.ts:22`, `api/auth/signup.ts:22`) e o logout (`api/auth/logout.ts`) apenas apaga o
cookie do navegador, sem invalidar o token no servidor.

## Evidencia
- api/auth/login.ts (arquivo inteiro)
- api/auth/login.ts:22, api/auth/signup.ts:22 (setExpirationTime('30d'))
- api/auth/logout.ts (nao invalida o token, so limpa o cookie local)

## Impacto
Forca bruta de senha sem restricao automatica. Um token vazado continua valido por ate 30 dias mesmo
apos logout ou troca de senha.

## Sugestao de correcao
Adicionar rate limiting por e-mail/IP no login. Implementar uma versao de sessao por usuario no banco,
checada a cada validacao de token, para permitir revogacao imediata.

## Criterios de aceite
- [ ] Login bloqueia temporariamente apos N tentativas erradas seguidas
- [ ] Logout invalida o token no servidor (nao so remove o cookie)
- [ ] Troca de senha invalida sessoes antigas
--- FIM ISSUE 5 ---
```